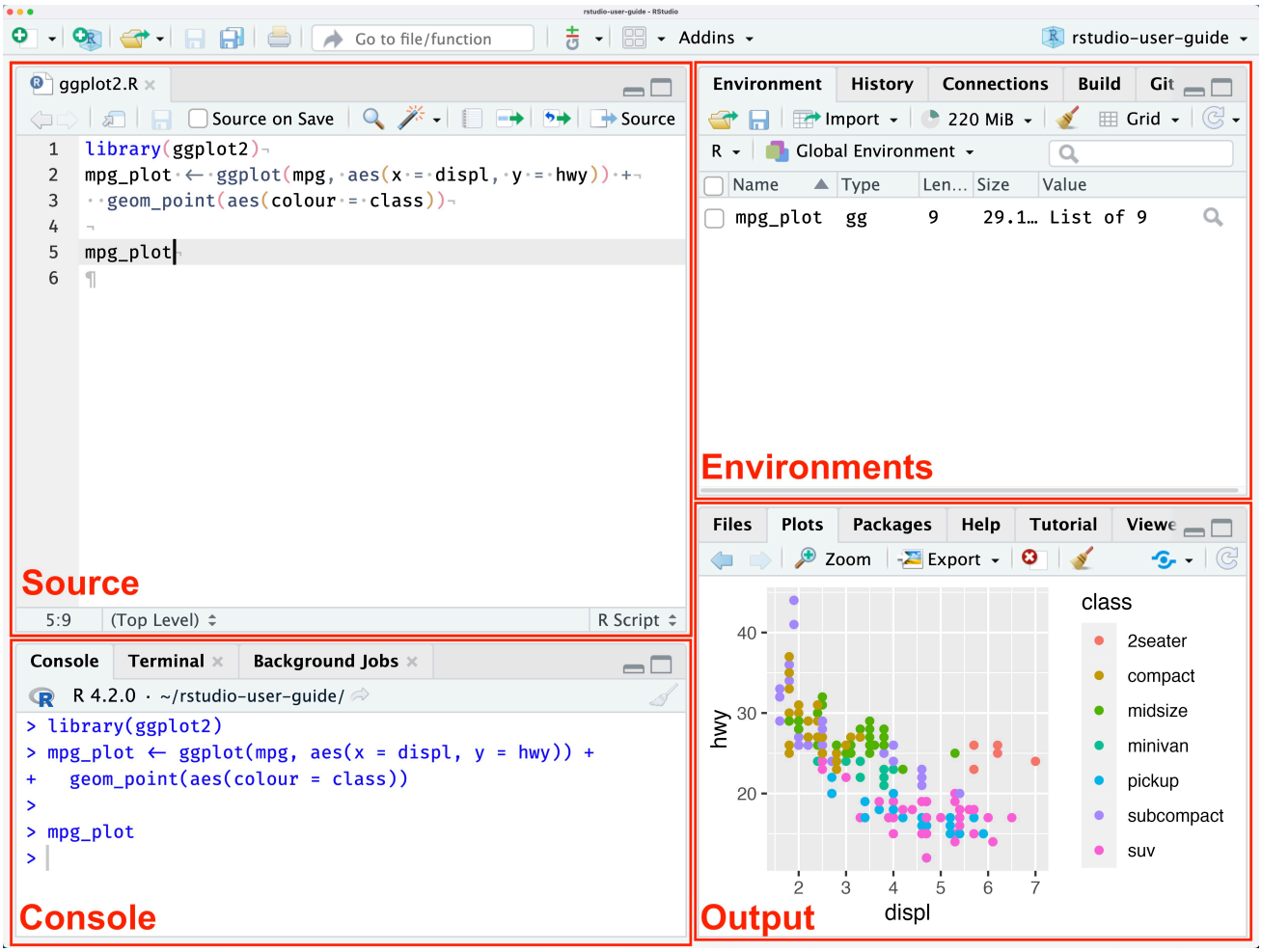
Introduction to R on Google Colab

R is a powerful programming language for data analysis, graphics, and especially statistical analysis. It is freely available to the public at www.r-project.org with easy-to-install binaries for Linux, macOS, and Windows. This notebook provides an introduction to R, designed for students and researchers. No prior experience is required, although some familiarity with scripting languages such as Bash, Matlab, or Python is helpful.

The best option for programming in R is to install RStudio or Visual Studio Code. Here, we'll learn some of the fundamentals of R and how to load and process data.



A Brief Introduction

When starting out in R (or any programming language), we learn three fundamental skills:

1. Reading Code: Initially, you'll encounter pre-built code. Reading is the first step to familiarizing yourself with the logic, commands, and how R "speaks."

- Example: understanding what `mean(c(1, 2, 3))` means.

2. Understanding Code: Here you begin to understand why that code is there, what it does, and how it connects to the data.

- Example: realizing that `mean()` calculates the mean and `c(1, 2, 3)` creates a vector (and what is a vector?)

3. Writing Code: The pinnacle of learning! With practice, you'll begin to write your own scripts and functions—whether for graphing, analysis, or automating tasks.

- Example: creating a graph with `plot(x, y)` and adjusting the parameters however you like.

Comments

Comments are parts of a computer program used to describe a piece of code. We can make a comment using the # symbol.

Purpose of comments? Readability, ignoring some code and pseudocode

In [1]:
```
# Anything that starts with # is a comment and is not executed
# This is used to explain what the code does
# or to leave notes or reminders
# or to disable a line of code
# or to make the code more readable or organized
```

Variable

Different types of values, such as characters, numbers, or logical values, can be assigned to a variable.

In [2]:
```
x <- 10 # The most common form of assignment
y = 20 # Also works
```

In R (and in virtually all programming languages), spaces in variable names are not allowed. If you try to write:

In [ ]:
```
patient age <- 25
```

R will throw an error because it will think you're trying to use the age variable, call the do function, access the patient...

a complete mess!

You can use the underscore (_), the period (.), or camelCase to separate words in variable names, like this:

In [3]:
```
patient_age <- 25
patient.age <- 25
PatientAge <- 25
```

In other words, whenever you create a variable, avoid spaces and use legal characters. Note that although each spelling is acceptable, each creates a different variable.

R differentiates EVERYTHING

When we speak or write with other people, we understand small errors, language variations, and misspellings. R is not like that.

In R, "age," "Age," "ages," and "Ages" are completely different names. This is because the language is case-sensitive.

And each variable must be unique within the same workspace; that is, each time I write a different way, I create a new object. This is important because it avoids confusion and ensures that you know exactly which variable you're using.

Therefore, create variable names that make sense, are easy to remember, and are different enough for YOU to work with.

Rules for naming variables:

- A variable name can be created using letters, digits, periods, and underscores.
- Start with a letter (never a number!)
- If a variable name starts with a period, you can't use digits after it.
- R (like other programming languages) is case-sensitive. This means that "age", "Age" and "AGE" are different variables.
- Avoid accents and special characters
- Be descriptive, but don't overdo it.
- Write simply and in a standardized format. The clearer your variable is, the easier it will be to reuse.

if_it_is_too_long_it_is_bad_to_reuse

What does `<-` mean?

This arrow is used to assign a value to an object. It's like saying, "Put this value here inside this little box." Or, "x is now worth 10 and y is now worth 20."

But the `<-` arrow is the traditional and most widely used form in the R community, especially in statistics and data science. It's like a language tradition—many say the arrow indicates where the value comes from and where it goes.

On the keyboard, to do "<-", you type "<" and then "-", and R automatically understands it as an assignment.

Data and Data Types

In R, data is typically organized into tables (data frames), which resemble Excel spreadsheets. Each column represents a variable (age, name, height, etc.) and each row represents an observation (a person, an animal, an experiment, etc.).

You can create data manually:

Type	Example	Description
Numeric	x <- 3.14	Numbers with or without decimals (e.g., 10, 2.5)
Integer	x <- 5L	Whole numbers (the L indicates "integer")
Character	city <- "Salvador"	Text (also called string)
Logical	active <- TRUE	True or false (TRUE or FALSE)
Factor	Sex <- factor(c("F", "M", "F"))	Categorical values (widely used in statistics)
Complex	z <- 2 + 3i	Complex numbers with imaginary part

| Date | today <- as.Date("2025-06-21") | Dates recognized as their own type

In [4]:
```r
# There are several data types in R, but the most common are:
# Numbers
x <- 10.5 # Decimal numbers

# Integer
y <- 5L # Integer numbers

# Text (we call them "strings")
name <- "my name is:" # Text in quotes (fill in your name)

# Logical (boolean)
true <- TRUE # True
false <- FALSE # False

# Vectors
ages <- c(25, 30, 35) # Vector of numbers
```

The `c()` is a function that stands for "combine" or "concatenate," used to create vectors.

All elements of a vector must be of the same type. If you don't use `c()`, R won't understand that you want to create a vector and will throw an error. When you use `c()`, R understands that you are combining these values into a single object, forming a vector.

When to use `c()`?

Whenever you want to create a vector with MULTIPLE values, whether numbers, text, or logical values.

Data frame

A data frame is like an Excel spreadsheet: each column is a variable and each row is an observation. You can import your own data or build it within R.

To build it:

In [5]:
```r
my_dataframe <- data.frame(
  Name = c("Camila", "Matias", "Juan"),
  Age = c(21,33,19),
  Student = c(TRUE, FALSE, TRUE)
)

my_dataframe

my_dataframe$Name # Access the "Name" column
```

A data.frame: 3 × 3

Name	Age	Student
<chr>	<dbl>	<lgl>
Camila	21	TRUE
Matias	33	FALSE
Juan	19	TRUE

'Camila' · 'Matias' · 'Juan'

Matrix

A matrix is a table of only numbers (or a single type of data), organized into rows and columns, as in mathematics.

In [6]:
```r
m <- matrix(1:9, nrow = 3, ncol = 3) # Create a 3x3 matrix, nrows = rows, ncol = columns

m
```

A matrix:
3 × 3 of
type int

1	4	7
2	5	8
3	6	9

Strings

In programming, strings are sequences of characters, that is, any set of letters, numbers, symbols, or spaces. They are used to represent text.

In [7]:
```r
my_string1 <- "my_string1"
```

```r
# Finding the length of a string
nchar(my_string1)

# Joining strings together
paste(my_string1, "2023")

#Splitting strings
txt<-("R is the statistical analysis language")
unlist(strsplit(txt, split = " ")) # Split words by space
```

10

'my_string1 2023'

'R' · 'is' · 'the' · 'statistical' · 'analysis' · 'language'

Vector

A vector is a basic structure that stores a collection of elements of the same type—they can be numbers, text (strings), logical values (TRUE/FALSE), etc. Vectors can perform operations such as adding, filtering, comparing, etc.

Think of a vector as a neat little box that holds several identical items:

- A vector of numbers: `c(1, 2, 3, 4)`
- A vector of text: `c("apple", "banana", "orange")`
- A logical vector: `c(TRUE, FALSE, TRUE)`

In [8]:
```r
# Vector
my_vector1 <- c(1,2,3)
my_vector2 <- c(4,5,6)
another_vector <- c("Camila", "Matias", "Juan")

class(my_vector1)
my_vector1
```

'numeric'

1 · 2 · 3

In [9]:
```r
class(another_vector)
another_vector
```

'character'

'Camila' · 'Matias' · 'Juan'

In [10]:
```r
# Creating vectors with R functions
my_name <- rep("Alejandra", times = 5) # Repeat "Alejandra" 5 times
my_name

my_seq <- rep(c(1,3,5), times = 3) # Repeat the sequence 3 times
my_seq
```

```r
my_seq <- rep(c(1,3,5), each = 3) # Repeat each element 3 times
my_seq

my_seq <- -10:10 # Sequence from -10 to 10
my_seq

my_seq <- seq(from = -10, to = 10, by = 2) # Sequence from -10 to 10 by 2
my_seq
```

'Alejandra' · 'Alejandra' · 'Alejandra' · 'Alejandra' · 'Alejandra'

1 · 3 · 5 · 1 · 3 · 5 · 1 · 3 · 5

1 · 1 · 1 · 3 · 3 · 3 · 5 · 5 · 5

-10 · -9 · -8 · -7 · -6 · -5 · -4 · -3 · -2 · -1 · 0 · 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10

-10 · -8 · -6 · -4 · -2 · 0 · 2 · 4 · 6 · 8 · 10

List

A list is a "pocket" that can hold anything: numbers, text, vectors, data frames, even functions!

Think of a shopping list:

1. Soap
2. Fabric softener
3. Apple
4. Orange
5. Biscuits
6. Bread
7. Olive oil
8. Chicken
9. Shampoo
10. Soap

Notice that the items aren't part of the same categories (food, cleaning supplies, hygiene products), but they're all in the same list.

It works similarly in R:

In [11]:
```r
my_list <- list(City = "Salvador", Age = 476, Beaches = c("Barra", "Itapuã", "Piatã", "Ribeira

my_list

my_list$City # Access the "City" element
```

$City	'Salvador'
$Age	476
$Beaches	'Barra' · 'Itapuã' · 'Piatã' · 'Ribeira'

'Salvador'

Factor

A factor is used to represent categorical variables, such as "sex," "marital status," or "yes/no response." Behind the scenes, R treats these values as named numeric levels.

In [12]:
```
sex <- factor(c("F", "M", "F"))
```

R understands this as a variable with two levels: "F" and "M."

Factors are essential in statistical analysis because they treat categories as distinct levels, not just any text.

Summary

Structure Type	What is it	Simple Example
Data Frame	Table with columns (variables) and rows (observations)	data <- data.frame(name, age)
Matrix	Table with only one type of data (all numbers, for example)	vector <- c(1,2,3)
String	Characters	txt <-("Hello, world!")
Vector	Collection of only one type of object	matrix(1:6, nrow = 2)
List	A collection of different objects (text, vectors, functions...)	list <- list(name, age, grades)
Factor	Categorical variable with defined levels	factor(c("yes", "no", "yes"))

Operations in R

R is a language created by mathematicians to solve statistical and mathematical problems. Therefore, performing operations in this language is quite similar to what we already know in traditional mathematics.

In [13]:
```
my_result <- 2 + 3

1 - my_result

6 * 4

2 ^ 3

10 / 5
```

```
log10(1000)

log2(32)

sqrt(144)
```

-4
24
8
2
3
5
12

Vector Operations

In [14]:
```
my_vector1
my_vector2
```

1 · 2 · 3

4 · 5 · 6

In [15]:
```
# Finding the length of a vector
length(my_vector1)
```

3

In [16]:
```
# Math
my_vector1 * my_vector2
my_vector1 + my_vector2
my_vector1 - my_vector2
my_vector1 / my_vector2
```

4 · 10 · 18

5 · 7 · 9

-3 · -3 · -3

0.25 · 0.4 · 0.5

In [17]:
```
# Indexing is a way to select (include/exclude) particular elements from a variable
my_vector2
my_vector2[1]
my_vector2[-c(1,3)]
my_vector2[2:3]
my_vector2[c(1,3)] # First and third only
```

4 · 5 · 6

4

5

5 · 6

4 · 6

In [18]:
```
# Create a vector with some random values
some_values <- c(8, 6, 1, 12, 3)
```

In [19]:
```
# Mean of the values
mean(some_values)
```

6

In [20]:
```
# Median of the values
median(some_values)
```

6

In [21]:
```
# Order the values
sort(some_values)
```

1 · 3 · 6 · 8 · 12

In [22]:
```
# Quartiles  (25% vs 75%)
quantile(some_values, 0.25)
quantile(some_values, 0.75)
```

25%: 3

75%: 8

In [23]:
```
# Interquartile range
IQR(some_values)
```

5

In [24]:
```
# Standard deviation
# In R, the standard deviation and the variance are computed
# as if the data represent a sample (n - 1)
sd(some_values)
```

4.30116263352131

In [25]:
```
# Variance
var(some_values)
```

18.5

Packages (or libraries)

Packages are collections of ready-made functions that help you perform specific tasks. For example:

- ggplot2: for elegant graphs
- dplyr: for manipulating data

- readr: for reading files

Before using a package, you need:

In [ ]:
```
# 1. Instalar (apenas uma vez):
install.packages("ggplot2")
```

In [27]:
```
# 2. Carregar (toda vez que vocês for usar):
library(ggplot2)
```

What are functions?

Functions are ready-made commands that do something for you. You've seen some above. They follow the structure:

In [ ]:
```
function_name(argument 1, argument 2, ...)
```

In [29]:
```
#Example:
mean(c(1, 2, 3, 4))
```

2.5

`mean` is the name of the function.

`c(1, 2, 3, 4)` is the argument (an array of numbers).

Functions save time and organize what you need to do. There are hundreds of them in R—and you can even create your own or use others'.

What are function arguments?

When we use a function in R, we need to pass it the necessary information so it knows what to do. This information is called arguments—think of them as the steps in an experiment.

In [30]:
```
round(3.14159, digits = 2)
```

3.14

In this line of code:

- `round()` is the function that rounds numbers.
- `3.14159` is the number to be rounded.
- `digits = 2` tells the function that we want 2 decimal places in the result.

Arguments by Position You can also pass the arguments in the order expected by the function:

In [31]:
```
round(3.14159, 2)
```

3.14

- R understands that the first value (3.14159) is the number to be rounded.
- The second value (2) is the number of decimal places.

This works because we're following the default order the function expects.

Named Arguments

You can also write the argument names explicitly—in which case, the order doesn't matter:

In [32]:
```
round(digits = 2, x = 3.14159)
```

3.14

This example does exactly the same thing, only now it's clearer what each value means:

digits = 2 → number of decimal places

x = 3.14159 → the number to be rounded

Why does this difference matter? When we're learning new functions—especially those with multiple arguments—it's much safer and clearer to use named arguments. This helps:

- better understand what each part of the code does
- avoid confusion if we forget the exact order
- make the script more readable for others (or for us in the future!).

Plotting Numerical Data

There are several ways to create graphs in R, but in this lesson we'll focus on the `ggplot2` package, a powerful and elegant tool for data visualization. It's based on the so-called grammar of graphs, which allows you to build graphs in a structured and intuitive way—as if you were writing meaningful sentences.

The grammar of graphs is a set of rules that defines how to build visualizations in a logical and standardized way. The great advantage is that, by following this grammar, you learn a single structure that serves to create different types of graphs—without having to memorize different arguments for each one.

This package stands out because it gives you complete control over the graph: you decide every part as axes, colors, visualization types, titles, legends... Everything can be added and adjusted with layers. This makes your graphs more personalized and informative.

The three main components of the grammar of graphs are:

- Data: the observations in our dataset.
- Aesthetics: Mappings of data to visual properties (such as axes and sizes of geometric objects).
- Geometries: Geometric objects, such as lines, that represent what we see in the graph.

Useful Resources:

PDF of the documentation: https://cran.r-project.org/web/packages/ggplot2/ggplot2.pdf

Here's the link to the website: https://ggplot2.tidyverse.org/

And here's the link to the book: https://ggplot2-book.org/

In [ ]:
```
# Load the tidyverse package
library(tidyverse)
```

In [34]:
```
# Load the msleep dataset
data(msleep)

# Check the structure of the msleep dataset
str(msleep)
```

```
tibble [83 × 11] (S3: tbl_df/tbl/data.frame)
 $ name        : chr [1:83] "Cheetah" "Owl monkey" "Mountain beaver" "Greater short-tailed shre
w" ...
 $ genus       : chr [1:83] "Acinonyx" "Aotus" "Aplodontia" "Blarina" ...
 $ vore        : chr [1:83] "carni" "omni" "herbi" "omni" ...
 $ order       : chr [1:83] "Carnivora" "Primates" "Rodentia" "Soricomorpha" ...
 $ conservation: chr [1:83] "lc" NA "nt" "lc" ...
 $ sleep_total : num [1:83] 12.1 17 14.4 14.9 4 14.4 8.7 7 10.1 3 ...
 $ sleep_rem   : num [1:83] NA 1.8 2.4 2.3 0.7 2.2 1.4 NA 2.9 NA ...
 $ sleep_cycle : num [1:83] NA NA NA 0.133 0.667 ...
 $ awake       : num [1:83] 11.9 7 9.6 9.1 20 9.6 15.3 17 13.9 21 ...
 $ brainwt     : num [1:83] NA 0.0155 NA 0.00029 0.423 NA NA NA 0.07 0.0982 ...
 $ bodywt      : num [1:83] 50 0.48 1.35 0.019 600 ...
```

In [35]:
```
nrow(msleep)
ncol(msleep)
head(msleep)
```

```
83
11
```

A tibble: 6 × 11

name	genus	vore	order	conservation	sleep_total	sleep_rem	sleep_cycle	awake	bra
<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<
Cheetah	Acinonyx	carni	Carnivora	lc	12.1	NA	NA	11.9	
Owl monkey	Aotus	omni	Primates	NA	17.0	1.8	NA	7.0	0.0
Mountain beaver	Aplodontia	herbi	Rodentia	nt	14.4	2.4	NA	9.6	
Greater short-tailed shrew	Blarina	omni	Soricomorpha	lc	14.9	2.3	0.1333333	9.1	0.0
Cow	Bos	herbi	Artiodactyla	domesticated	4.0	0.7	0.6666667	20.0	0.4
Three-toed sloth	Bradypus	herbi	Pilosa	NA	14.4	2.2	0.7666667	9.6	

In [36]:
```
# Are there NA values? How many?
any(is.na(msleep))
sum(is.na(msleep))
```

TRUE

136

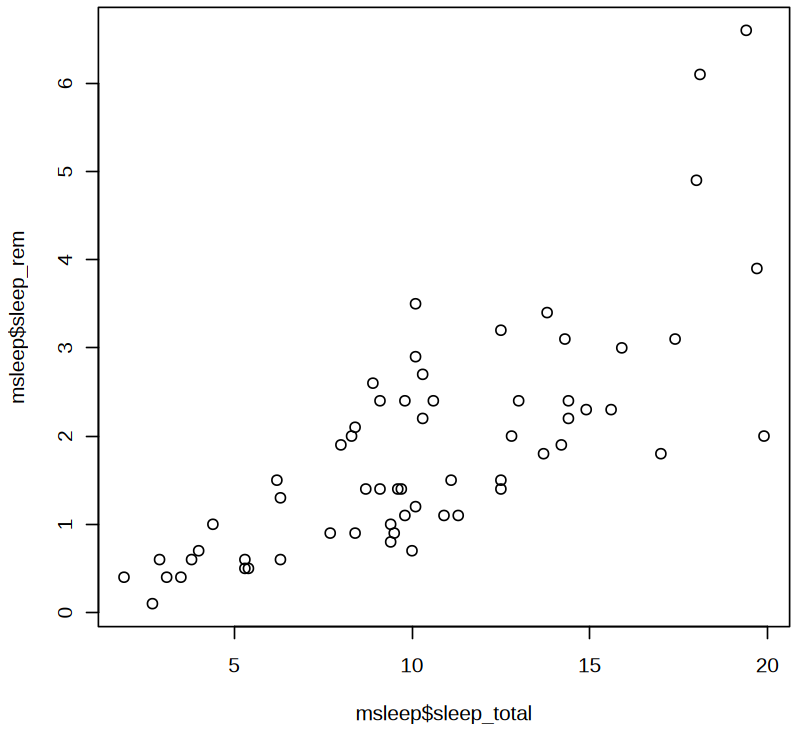
In [37]:
```
# Leave out any observation with NA values
msleep %>%
  drop_na()

#Equal to do:
#drop_na(msleep)
```

A tibble: 20 × 11

name	genus	vore	order	conservation	sleep_total	sleep_rem	sleep_cycle	awak
<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl
Greater short-tailed shrew	Blarina	omni	Soricomorpha	lc	14.9	2.3	0.1333333	9.
Cow	Bos	herbi	Artiodactyla	domesticated	4.0	0.7	0.6666667	20.
Dog	Canis	carni	Carnivora	domesticated	10.1	2.9	0.3333333	13.
Guinea pig	Cavis	herbi	Rodentia	domesticated	9.4	0.8	0.2166667	14.
Chinchilla	Chinchilla	herbi	Rodentia	domesticated	12.5	1.5	0.1166667	11.
Lesser short-tailed shrew	Cryptotis	omni	Soricomorpha	lc	9.1	1.4	0.1500000	14.
Long-nosed armadillo	Dasypus	carni	Cingulata	lc	17.4	3.1	0.3833333	6.
North American Opossum	Didelphis	omni	Didelphimorphia	lc	18.0	4.9	0.3333333	6.
Big brown bat	Eptesicus	insecti	Chiroptera	lc	19.7	3.9	0.1166667	4.
Horse	Equus	herbi	Perissodactyla	domesticated	2.9	0.6	1.0000000	21.
European hedgehog	Erinaceus	omni	Erinaceomorpha	lc	10.1	3.5	0.2833333	13.
Domestic cat	Felis	carni	Carnivora	domesticated	12.5	3.2	0.4166667	11.
Golden hamster	Mesocricetus	herbi	Rodentia	en	14.3	3.1	0.2000000	9.
House mouse	Mus	herbi	Rodentia	nt	12.5	1.4	0.1833333	11.
Rabbit	Oryctolagus	herbi	Lagomorpha	domesticated	8.4	0.9	0.4166667	15.
Laboratory rat	Rattus	herbi	Rodentia	lc	13.0	2.4	0.1833333	11.
Eastern american mole	Scalopus	insecti	Soricomorpha	lc	8.4	2.1	0.1666667	15.
Thirteen-lined	Spermophilus	herbi	Rodentia	lc	13.8	3.4	0.2166667	10.

name	genus	vore	order	conservation	sleep_total	sleep_rem	sleep_cycle	awak
<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl
ground squirrel								
Pig	Sus	omni	Artiodactyla	domesticated	9.1	2.4	0.5000000	14.
Brazilian tapir	Tapirus	herbi	Perissodactyla	vu	4.4	1.0	0.9000000	19.

In [38]:
```
# Is there a relationship between sleep total and sleep rem?
# How can we check that?
# Plotting maybe?

plot(msleep$sleep_total, msleep$sleep_rem)
```



In [39]:
```
# Correlation coefficient can be computed using the functions cor()
cor.test(msleep$sleep_total, msleep$sleep_rem,
         method = "pearson", use = "complete.obs")
```

```
        Pearson's product-moment correlation

data:  msleep$sleep_total and msleep$sleep_rem
t = 8.7564, df = 59, p-value = 2.916e-12
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6166756 0.8438320
sample estimates:
     cor
0.751755
```

In [40]:
```r
# Correlation coefficient can be computed using the functions cor()
cor.test(msleep$sleep_total, msleep$sleep_rem,
         method = "pearson", use = "complete.obs")
```

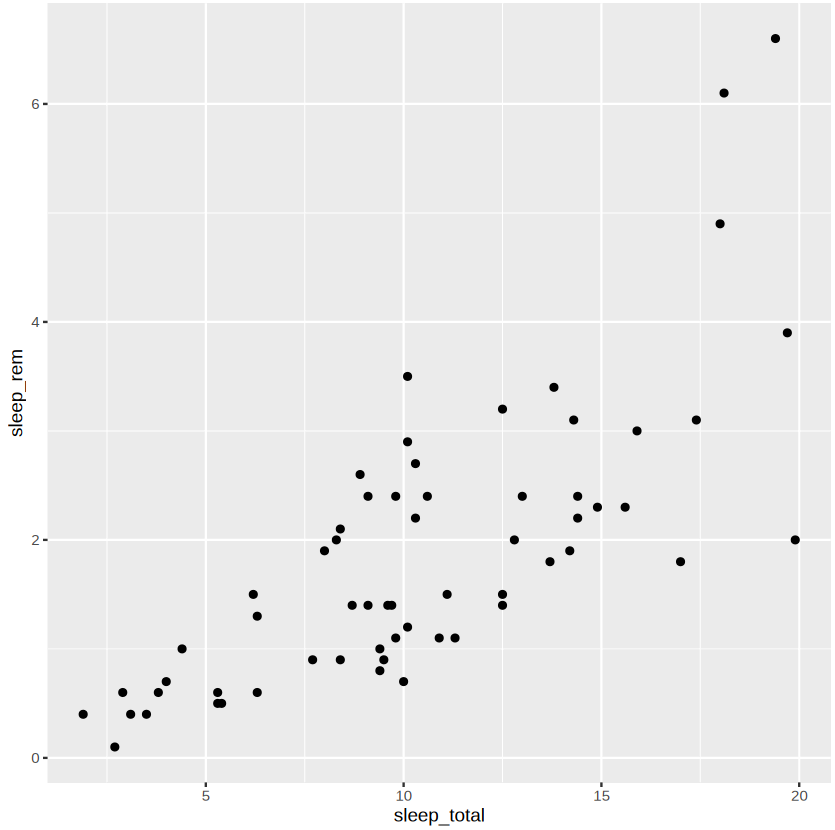
```
        Pearson's product-moment correlation

data:  msleep$sleep_total and msleep$sleep_rem
t = 8.7564, df = 59, p-value = 2.916e-12
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6166756 0.8438320
sample estimates:
     cor
0.751755
```

In [41]:
```r
#Comparing means
t.test(msleep$sleep_total, msleep$sleep_rem)
```

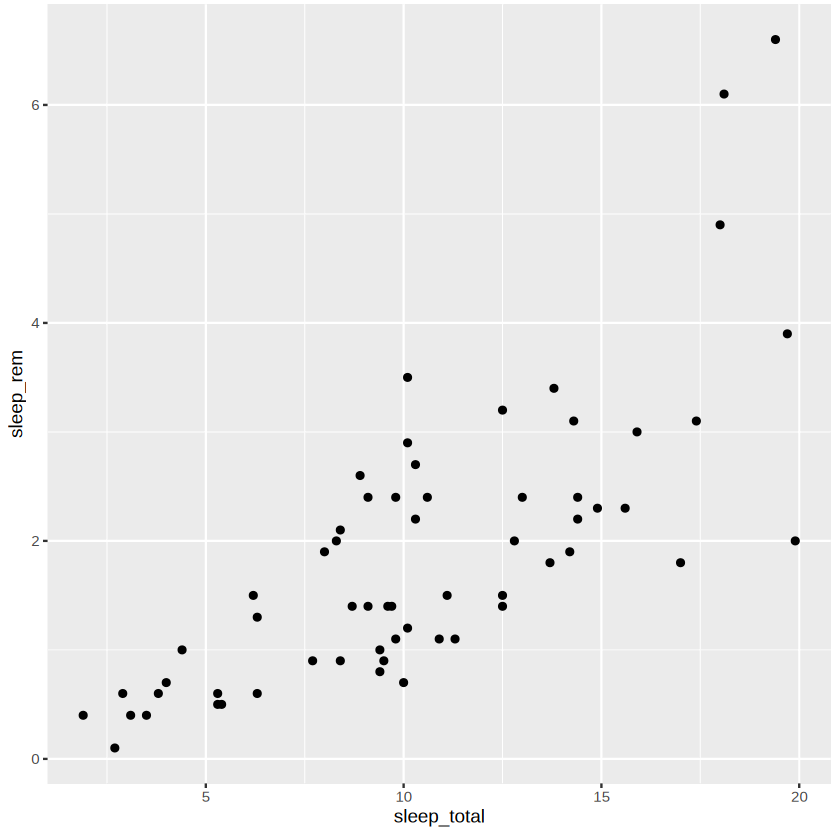
```
        Welch Two Sample t-test

data:  msleep$sleep_total and msleep$sleep_rem
t = 16.586, df = 100.25, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 7.53463 9.58202
sample estimates:
mean of x mean of y
 10.43373   1.87541
```

Applying the ggplot2 package

In [42]:
```r
# Using tidyverse
msleep %>% # %>% tideverse's operator for piping functions
  ggplot(aes(x = sleep_total, y = sleep_rem)) +
  geom_point()
```



In [43]:
```
#Directly calling ggplot
ggplot(msleep, aes(x = sleep_total, y = sleep_rem)) +
  geom_point()
```



In [44]:
```
#Saving plots
g <- ggplot(msleep, aes(x = sleep_total, y = sleep_rem)) +
```

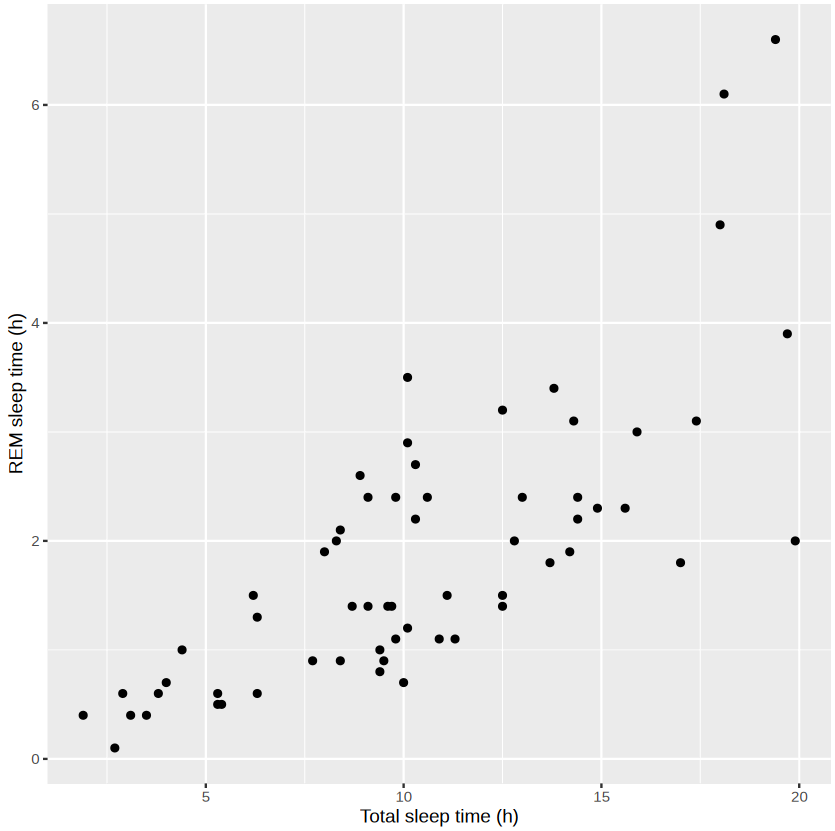
```
  geom_point()
ggsave(filename = "test.pdf", plot = g)
```

```
Saving 6.67 x 6.67 in image
```

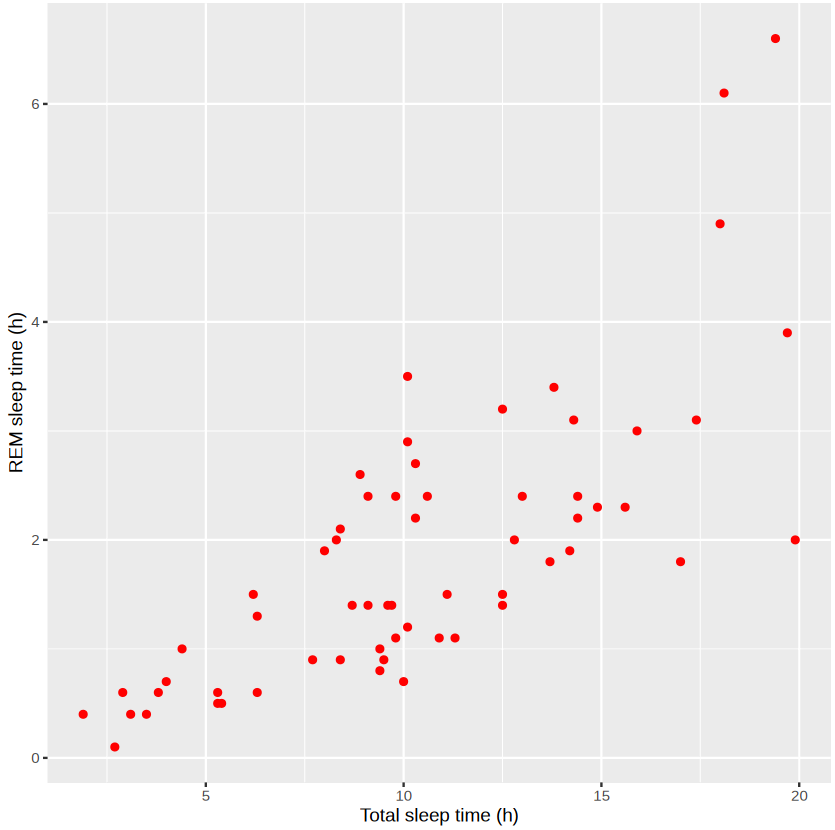
In [45]:
```
#Other alternative
pdf("test2.pdf") # Open the PDF device
g # Print the plot to the PDF device
dev.off() # Close the PDF device
```

agg_record_190628847: 2

In [46]:
```
# Add a custom y-axis title
msleep %>%
  ggplot(aes(sleep_total, sleep_rem)) + # Implicit or explicit arguments?
  geom_point() +
  xlab("Total sleep time (h)") +
  ylab("REM sleep time (h)")
```



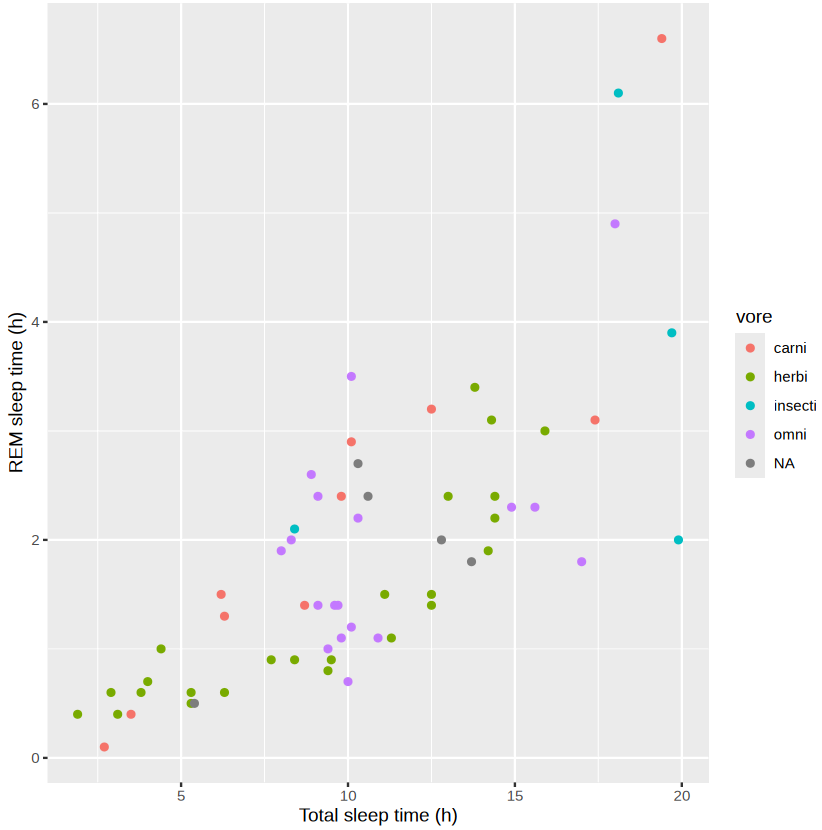
In [47]:
```
msleep %>%
  ggplot(aes(sleep_total, sleep_rem)) +
  geom_point(color = "red") +  # Visit: color-hex.com to select it
  xlab("Total sleep time (h)") +
  ylab("REM sleep time (h)")
```



In [48]:
```
# How can we colour the points by vore?
str(msleep)
```

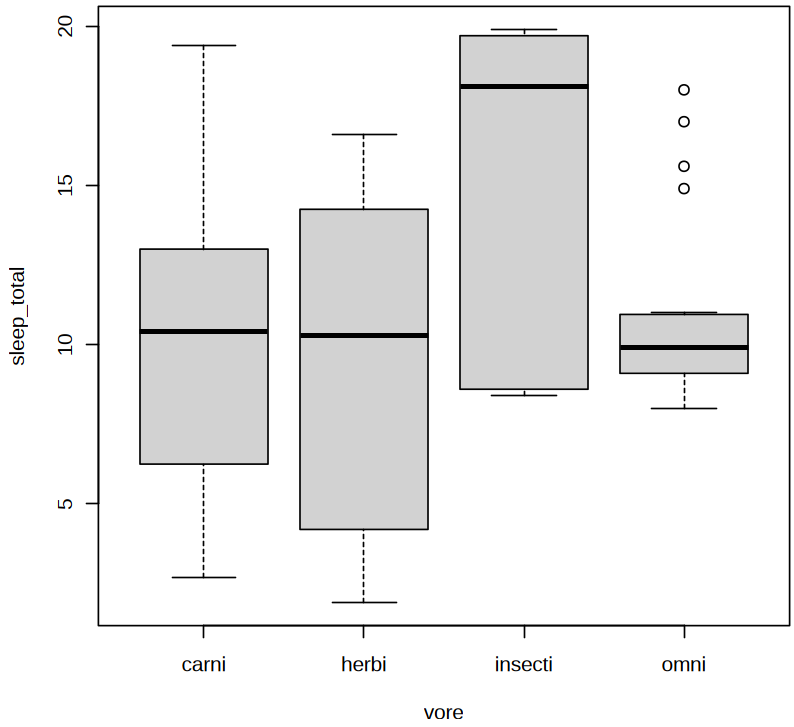
```
tibble [83 × 11] (S3: tbl_df/tbl/data.frame)
 $ name        : chr [1:83] "Cheetah" "Owl monkey" "Mountain beaver" "Greater short-tailed shre
w" ...
 $ genus       : chr [1:83] "Acinonyx" "Aotus" "Aplodontia" "Blarina" ...
 $ vore        : chr [1:83] "carni" "omni" "herbi" "omni" ...
 $ order       : chr [1:83] "Carnivora" "Primates" "Rodentia" "Soricomorpha" ...
 $ conservation: chr [1:83] "lc" NA "nt" "lc" ...
 $ sleep_total : num [1:83] 12.1 17 14.4 14.9 4 14.4 8.7 7 10.1 3 ...
 $ sleep_rem   : num [1:83] NA 1.8 2.4 2.3 0.7 2.2 1.4 NA 2.9 NA ...
 $ sleep_cycle : num [1:83] NA NA NA 0.133 0.667 ...
 $ awake       : num [1:83] 11.9 7 9.6 9.1 20 9.6 15.3 17 13.9 21 ...
 $ brainwt     : num [1:83] NA 0.0155 NA 0.00029 0.423 NA NA NA 0.07 0.0982 ...
 $ bodywt      : num [1:83] 50 0.48 1.35 0.019 600 ...
```

In [49]:
```
msleep %>%
  ggplot(aes(sleep_total, sleep_rem, color = vore)) +
  geom_point() +
  xlab("Total sleep time (h)") +
  ylab("REM sleep time (h)")
```

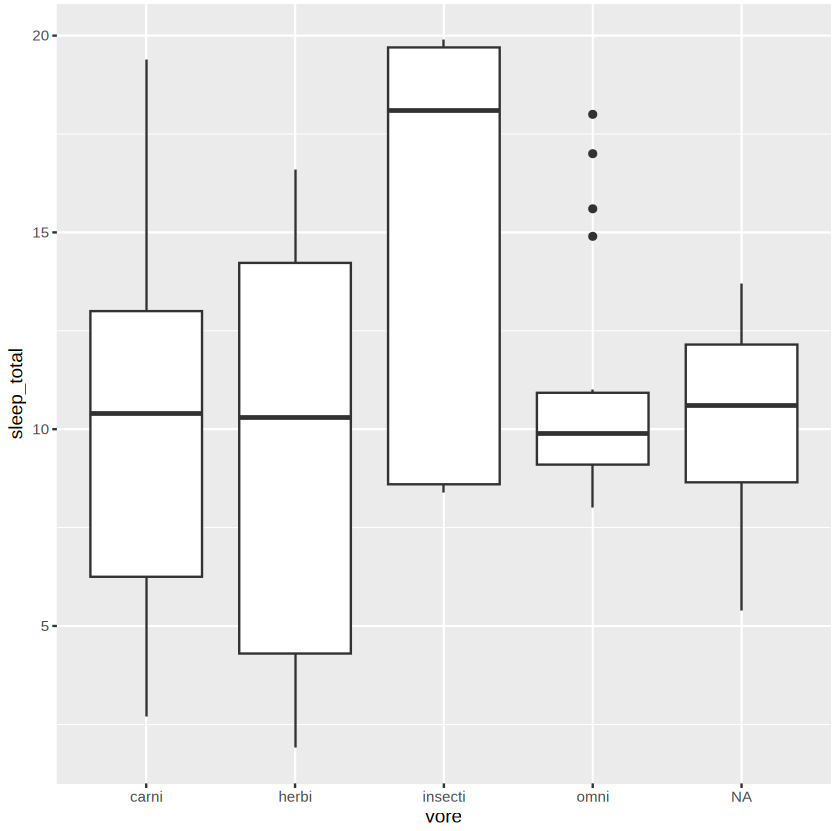


Boxplot

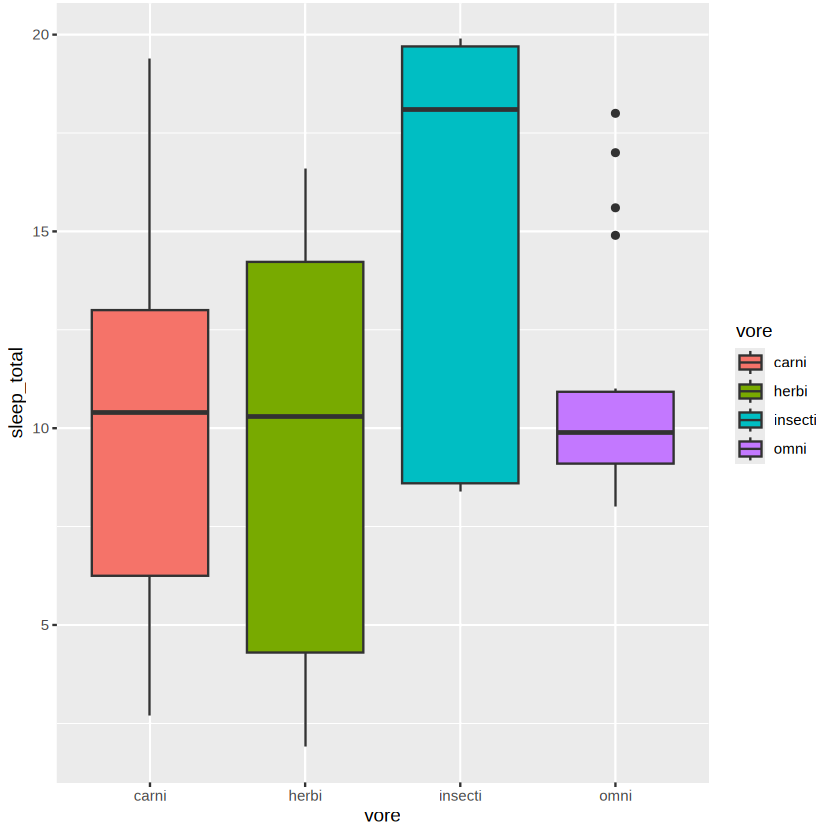
In [50]:
```
# Base R
boxplot(sleep_total ~ vore, data = msleep) # sleep_total by vore, the ~ means "by"
```



In [51]:
```
# ggplot2
msleep %>%
  ggplot(aes(vore, sleep_total)) +
      geom_boxplot()
```

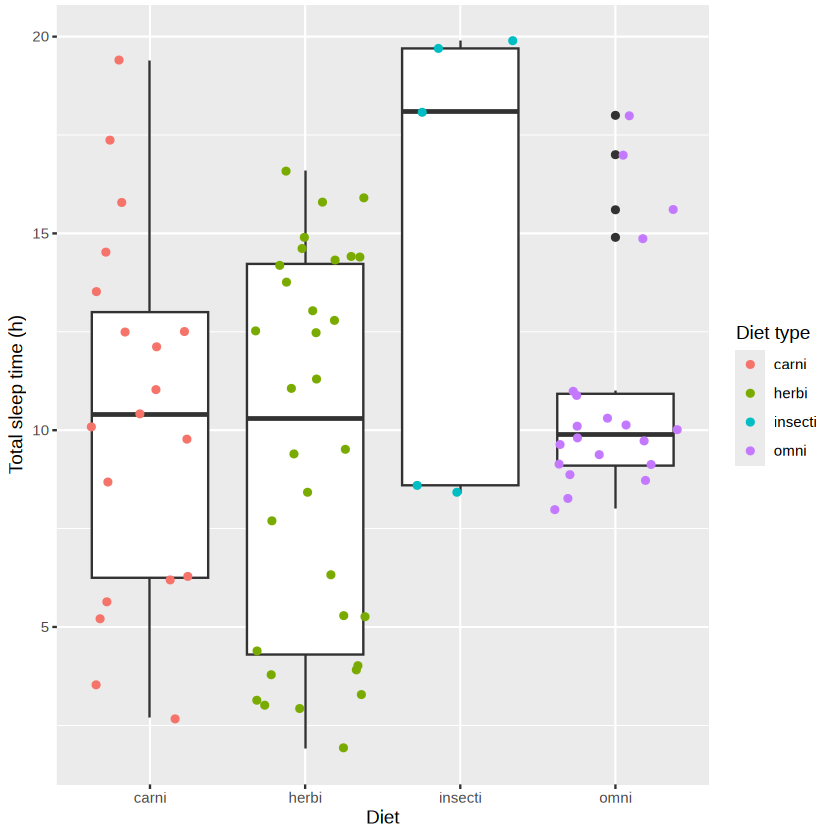


In [52]:
```
# Filtering out NAs
# Fill: collors based on vore
msleep %>%
  filter(!(is.na(msleep$vore))) %>%
  ggplot(aes(vore, sleep_total, fill = vore)) +
  geom_boxplot()
```



In [53]:

```
msleep %>%
  filter(!(is.na(msleep$vore))) %>%
  ggplot(aes(vore, sleep_total)) +
  geom_boxplot() +
  geom_jitter(aes(color=vore)) +
  labs(x= "Diet", y = "Total sleep time (h)", color ="Diet type")
```


Total sleep time (h)
20
15
10
5
carni
herbi
insecti
omni
Diet
Diet type
carni
herbi
insecti
omni